

Estudio comparativo de metaheurísticas para el Flexible Job Shop Scheduling Problem (FJSP)

Benjamín Velásquez, Alex Arancibia, Pablo Martínez, Martín Sáez, Felipe Oregón,
Gonzalo Anabalón, Kevin Muñoz y Maximiliano Soto

Ingeniería Civil Informática
Universidad Andrés Bello
Santiago, Chile

Autor: Gustavo Gatica
Universidad Andrés Bello
Santiago, Chile

Resumen—En el dinámico entorno industrial actual, la optimización de procesos no es una opción secundaria, sino el pilar fundamental para garantizar la competitividad y la sostenibilidad de cualquier fábrica productiva. El *Flexible Job Shop Scheduling Problem* (FJSP) es una generalización del problema clásico de job shop donde cada operación puede ejecutarse en una de varias máquinas elegibles. La *hardiness* del problema lo clasifica como NP-hard, de gran relevancia en la planificación de sistemas de manufactura flexibles. En este trabajo se implementan y comparan siete metaheurísticas —Búsqueda Tabú, TS+VNS, Algoritmo Genético, Algoritmo Memético, PSO, ACO (variante MMAS) y Simulated Annealing (SA)— aplicadas a un conjunto común de 31 instancias de referencia bajo un protocolo experimental uniforme, más una octava metaheurística (HHS) evaluada de forma independiente. Las instancias provienen de la literatura. Los resultados muestran diferencias claras de desempeño: el Algoritmo Memético lidera el ranking global con un gap de $-0,57\%$ respecto a las cotas de referencia, seguido por el SA ($-0,31\%$) y el AG ($+0,17\%$), mientras que el ACO presenta las mayores dificultades, especialmente en el benchmark de Dauzère ($+38,30\%$).

Index Terms—flexible job shop, metaheurísticas, makespan, búsqueda tabú, algoritmo genético, simulated annealing, colonia de hormigas, optimización combinatoria

I. INTRODUCCIÓN

La programación de la producción es un problema central en la ingeniería industrial y la investigación de operaciones [3]. En el dinámico entorno industrial actual, la capacidad de planificar eficientemente el uso de las máquinas incide directamente en la competitividad y la sostenibilidad de los sistemas de manufactura. El problema de job shop (JSP) consiste en ordenar un conjunto de operaciones sobre un conjunto de máquinas con el fin de minimizar el makespan, es decir, el tiempo de finalización de la última operación.

El *Flexible Job Shop Scheduling Problem* (FJSP) extiende el JSP clásico permitiendo que cada operación se ejecute en cualquiera de varias máquinas elegibles [1]. Esto introduce una segunda dimensión de decisión: además de secuenciar las operaciones (*problema de secuenciación*), hay que asignar cada una a una máquina concreta (*problema de ruteo*) [4]. Esta flexibilidad refleja de forma más realista los sistemas de manufactura modernos, pero también incrementa la complejidad

del problema. La dureza del problema lo clasifica como NP-duro [8], lo que implica que no existe algoritmo polinomial conocido capaz de resolverlo de forma exacta en instancias de tamaño realista.

La literatura presenta dos grandes enfoques para resolver el FJSP: métodos exactos, como la programación lineal entera mixta (MILP), que garantizan optimalidad pero solo son tratables en instancias pequeñas [9]; y metaheurísticas, que obtienen soluciones de buena calidad con esfuerzo computacional acotado [2]. Este trabajo implementa y compara siete metaheurísticas sobre un conjunto común de instancias —más una octava evaluada de forma independiente— bajo un protocolo experimental uniforme, con el objetivo de caracterizar sus fortalezas y debilidades relativas en el FJSP.

El resto del documento se organiza como sigue. La Sección II define formalmente el problema; la Sección III presenta el modelo matemático MILP; la Sección IV describe las metaheurísticas implementadas; la Sección V detalla el protocolo experimental y las instancias; la Sección VI reporta los resultados; y las Secciones VII y VIII discuten los hallazgos y presentan las conclusiones.

II. DEFINICIÓN DEL PROBLEMA

El FJSP puede describirse así: dados n trabajos y m máquinas, cada trabajo requiere ejecutar una secuencia de operaciones en orden estricto, y cada operación puede realizarse en cualquiera de un subconjunto de máquinas con tiempos de procesamiento posiblemente distintos. El objetivo es asignar cada operación a una máquina y determinar su tiempo de inicio minimizando el makespan. La Fig. 1 ilustra este concepto.

Formalmente, sea $J = \{J_1, \dots, J_n\}$ un conjunto de n trabajos que deben procesarse sobre $\mathcal{M} = \{1, \dots, m\}$ máquinas. Cada trabajo J_j consta de n_j operaciones $O_{1j}, O_{2j}, \dots, O_{n_jj}$ que deben ejecutarse en orden estricto. Cada operación O_{ij} puede asignarse a cualquier máquina k del subconjunto $M_{ij} \subseteq \mathcal{M}$, con tiempo de procesamiento $p_{ijk} > 0$ que puede variar según la máquina. El objetivo es minimizar el makespan $C_{\max}$, respetando que cada máquina procesa a lo sumo una operación

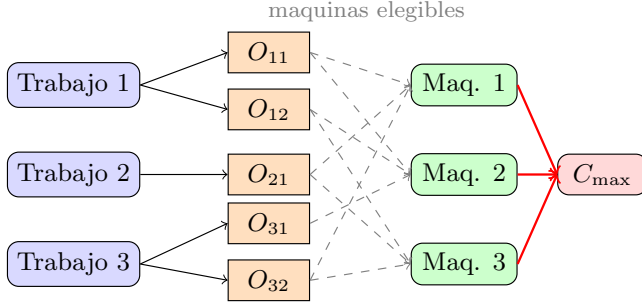


Figura 1: Esquema del FJSP: trabajos con operaciones asignables a máquinas elegibles (líneas punteadas). Objetivo: minimizar $C_{\max}$.

a la vez y que las operaciones de cada trabajo respetan su orden de precedencia.

III. MODELO MATEMÁTICO

El FJSP puede formularse como un programa lineal entero mixto (MILP) siguiendo la propuesta de Brandimarte [3], implementada en AMPL.

III-A. Variables de decisión

Las variables del modelo son: $x_{ijk} \in \{0,1\}$, que vale 1 si la operación j del trabajo i se asigna a la máquina k ; $s_{ij} \geq 0$, el tiempo de inicio de dicha operación; $y_{ijkab} \in \{0,1\}$, que ordena dos operaciones en competencia por una misma máquina; y $C_{\max} \geq 0$, el makespan a minimizar.

III-B. Formulación

$$\text{mín } C_{\max} \quad (1)$$

$$\text{s.a. } \sum_{k \in M_{ij}} x_{ijk} = 1 \quad \forall i \in I, j \in J_i \quad (2)$$

$$s_{ij} \geq s_{i,j-1} + \sum_k p_{i,j-1,k} x_{i,j-1,k} \quad (3)$$

$$s_{ij} + \sum_k p_{ijk} x_{ijk} \leq C_{\max} \quad (4)$$

$$s_{ij} \geq s_{ab} + p_{abk} - B(1 - y_{ijkab}) - B(2 - x_{ijk} - x_{abk}) \quad (5)$$

$$s_{ab} \geq s_{ij} + p_{ijk} - B y_{ijkab} - B(2 - x_{ijk} - x_{abk}) \quad (6)$$

donde $B = \sum_{i,j,k} p_{ijk}$ es la constante disyuntiva. La restricción (2) garantiza la asignación única de cada operación; (3) impone la precedencia intratrabajo; (4) define el makespan; y (5)–(6) evitan la solapación en cada máquina, activándose solo cuando ambas operaciones se asignan a la misma máquina k (términos x_{ijk} , x_{abk}). La implementación en AMPL sigue la estructura de Brandimarte [3] con conjuntos I , K , $J\{i \text{ in } I\}$ y $M\{i \text{ in } I, j \text{ in } J[i]\}$.

Dado que el modelo exacto solo es tratable para instancias pequeñas, el resto del trabajo se concentra en enfoques metaheurísticos.

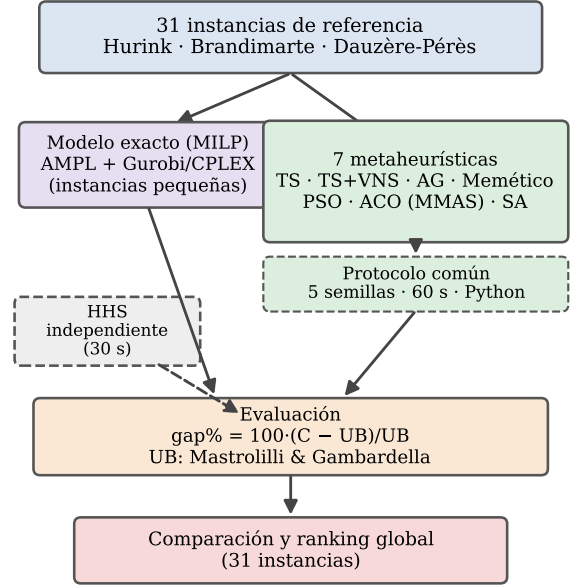


Figura 2: Metodología de trabajo: sobre un conjunto común de 31 instancias se aplican dos enfoques —un modelo exacto (MILP) y siete metaheurísticas bajo un protocolo uniforme—, evaluados mediante el gap % respecto a la cota UB . La HHS se evalúa de forma independiente.

IV. EXPERIMENTO COMPUTACIONAL

Cada integrante del grupo implementó una metaheurística distinta, todas aplicadas sobre el mismo problema e instancias. A continuación se describen brevemente. La Fig. 2 resume la metodología de trabajo.

Búsqueda Tabú (TS) [5]. Método de trayectoria que explora el espacio de soluciones moviéndose de vecino en vecino manteniendo una lista tabú para prohibir visitar soluciones recientes, lo que permite escapar de óptimos locales. Se complementa con un criterio de aspiración.

TS+VNS (híbrida) [6]. Combina Búsqueda Tabú con Búsqueda de Vecindad Variable (VNS). El VNS agita la solución incumbente cuando la TS se estanca; la TS realiza la intensificación local. Emplea tres vecindarios con agitación adaptativa.

Algoritmo Genético (AG) [10]. Método poblacional que evoluciona soluciones mediante operadores de selección, cruzamiento y mutación, favoreciendo a los individuos con menor makespan en cada generación.

Algoritmo Memético [11]. Híbrido que combina un AG con búsqueda local basada en elementos de TS. El refinamiento

Tabla I: Parámetros principales y criterio de parada por metaheurística. Todas comparten el límite de 60 s por ejecución.

Metaheur.	Parámetros principales	Criterio de parada
Tabú	Tenencia dinámica $\sim \sqrt{n_{op}}$; 3 vecindarios; ≤ 300 vecinos/iter	60 s o 40 iter. sin mejora
TS+VNS	Tenencia 10; $k_{m\acute{a}x} = 4$; 40 iteraciones	60 s
AG	Población 100; $p_c = 0,9$; $p_m = 0,2$; torneo $k = 3$; elitismo 2	60 s o 120 gen. sin mejora
Memético	Población 40; $p_c = 0,9$; $p_m = 0,2$; elitismo 2; búsqueda local (30 pruebas, $p_{ls} = 0,5$)	60 s
PSO	Enjambre 40; $w : 0,9 \rightarrow 0,4$; $c_1 = 1,6$; $v_{m\acute{a}x} = 0,3$	60 s o 60 iter. sin mejora
ACO (MMAS)	30 hormigas; $\alpha = 1,0$; $\beta = 2,0$; $\rho = 0,1$; búsqueda local	60 s
SA	(por completar)	60 s

to local de cada individuo acelera la convergencia respecto a un AG puro.

Optimización por Enjambre de Partículas (PSO) [12]. Método poblacional donde un conjunto de partículas se desplaza por el espacio de soluciones, guiadas por su mejor posición individual histórica y la mejor posición global del enjambre.

Optimización por Colonia de Hormigas (ACO, MMAS) [13], [14]. Método poblacional que construye soluciones probabilísticamente guiado por rastros de feromona. La variante MMAS [14] acota los valores de feromona entre un límite mínimo y máximo para evitar convergencia prematura. Se utilizó una tasa de evaporación de feromona del 10 % por iteración ($\rho = 0,1$), conservando el 90 % del rastro acumulado entre iteraciones, siguiendo las reglas del MAX-MIN Ant System [14].

Simulated Annealing (SA) [15]. Método de trayectoria inspirado en el recocido metalúrgico. Un vecino aleatorio se acepta si mejora la solución actual; si la empeora, se acepta con probabilidad $e^{-\Delta/T}$, donde Δ es el deterioro y T la temperatura, que decrece según un esquema predefinido.

Búsqueda Armónica Híbrida (HHS) [16]. Variante híbrida de la Búsqueda Armónica, inspirada en la improvisación musical. Una memoria armónica almacena las mejores soluciones y las nuevas se construyen combinando componentes de dicha memoria con perturbaciones aleatorias. Esta metaheurística fue incorporada con posterioridad al protocolo experimental, por lo que sus resultados se presentan de forma independiente (Sección VI-A).

En cuanto a la representación del problema, la mayoría de los métodos poblacionales (AG, Memético, PSO) emplea una codificación de dos vectores —secuencia de operaciones y asignación de máquinas—, mientras que los métodos de trayectoria (TS, TS+VNS, SA) operan directamente sobre la solución mediante vecindarios. La Tabla I resume los parámetros principales y el criterio de parada de cada metaheurística.

Tabla II: Instancias seleccionadas por benchmark.

Benchmark	Inst.	T	M	Criterio
Hurink (e/r/vdata)	la01	10	5	3 niveles de flexibilidad; 10×5 a 15×10 .
	la06	15	5	
	la11	20	5	
	la16	10	10	
	la21	15	10	
Brandimarte	Mk01–10	10–20	4–15	Las 10 instancias completas.
Dauzère-Pérès	01a	10	5	2 inst. por grupo (10×5 , 15×8 , 20×10).
	04a	10	5	
	07a	15	8	
	10a	15	8	
	13a	20	10	
	16a	20	10	

V. CONFIGURACIÓN COMPUTACIONAL

V-A. Protocolo común

Para garantizar la comparabilidad, todos los integrantes adoptaron el mismo protocolo:

- **Repeticiones:** 5 ejecuciones independientes por instancia.
- **Semillas:** 100, 101, 102, 103, 104 (fijas).
- **Tiempo límite:** 60 s por ejecución.
- **Métrica:** $\text{gap \%} = 100 \cdot (C_{\text{mejor}} - UB) / UB$, con UB de Mastrolilli & Gambardella [2].
- **Lenguaje:** Python.

V-B. Instancias de prueba

Las 31 instancias provienen de tres benchmarks clásicos del FJSP, resumidos en la Tabla II. Las cotas superiores de referencia fueron tomadas de Mastrolilli & Gambardella [2]. El benchmark de Barnes & Chambers [7] se excluyó por ser el menos referenciado en la literatura reciente de FJSP.

V-C. Justificación de la selección de instancias

Los tres benchmarks seleccionados son los más utilizados en la literatura de FJSP y fueron adoptados por Mastrolilli & Gambardella [2], garantizando comparabilidad directa con trabajos previos. De Hurink se incluyeron las tres variantes de flexibilidad para estudiar su efecto, seleccionando cinco instancias de 10×5 a 15×10 . Brandimarte se incluyó íntegro (Mk01–Mk10) por ser el benchmark más citado en comparaciones de FJSP. La Fig. 3 ilustra, de forma esquemática, la estructura de salida de una solución para estas instancias: cada metaheurística entrega una asignación de operaciones a máquinas representable como un diagrama de Gantt.

De Dauzère-Pérès se seleccionaron dos instancias por grupo de tamaño para representar la variabilidad sin incurrir en tiempo de cómputo excesivo. Barnes & Chambers [7] se excluyó por ser el menos referenciado en la literatura reciente.

VI. RESULTADOS

La Tabla IV presenta el $\text{gap \%}$ del mejor makespan (BKS sobre 5 repeticiones) respecto a la cota UB para cada benchmark. Valores negativos indican que la metaheurística superó

Tabla III: Mejor makespan (mín. sobre 5 repeticiones) por instancia, para las ocho metaheurísticas evaluadas. *HHS usa un protocolo independiente (30 s/ejec.) y no es directamente comparable.

Bench.	Inst.	UB	Memét.	SA	AG	TS+VNS	PSO	Tabú	ACO	HHS*
Hurink edata	la01	609	609	609	609	609	609	621	644	612
	la06	800	833	833	833	833	833	866	859	836
	la11	1071	1104	1104	1106	1115	1134	1122	1218	1114
	la16	717	892	915	892	918	915	923	1010	921
	la21	835	1070	1071	1086	1100	1078	1162	1292	1120
Hurink rdata	la01	570	573	580	584	587	598	577	620	578
	la06	799	799	804	801	805	812	804	850	803
	la11	1071	1071	1072	1076	1073	1076	1079	1126	1079
	la16	717	717	730	741	755	773	761	832	734
	la21	833	872	897	945	948	992	976	1132	966
Hurink vdata	la01	570	571	574	572	578	581	573	602	575
	la06	799	800	806	802	807	812	804	832	806
	la11	1071	1071	1073	1076	1076	1079	1076	1125	1080
	la16	717	717	717	717	717	735	717	754	717
	la21	800	861	861	934	902	956	895	968	959
Brand.	Mk01	42	40	40	40	42	42	40	42	41
	Mk02	32	27	29	28	29	29	28	36	29
	Mk03	211	204	204	204	204	204	204	236	204
	Mk04	81	61	61	60	67	68	66	75	65
	Mk05	186	174	177	174	178	177	179	191	176
	Mk06	86	64	63	65	73	72	70	103	73
	Mk07	157	143	150	144	153	150	147	180	146
	Mk08	523	523	523	523	523	523	523	566	523
	Mk09	369	307	307	307	330	333	322	423	325
	Mk10	296	230	224	228	246	247	250	337	273
Dauzère	01a	2530	2593	2574	2610	2568	2675	2778	3192	2750
	04a	2565	2602	2565	2576	2575	2655	2762	3147	2714
	07a	2408	2506	2474	2470	2582	2665	2739	3329	2711
	10a	2362	2549	2475	2488	2563	2554	2760	3349	2789
	13a	2302	2560	2498	2533	2638	2714	2782	3439	2719
	16a	2301	2550	2470	2517	2618	2665	2794	3491	2698

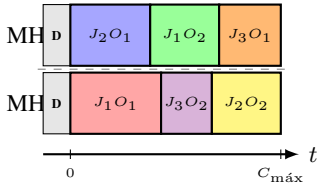


Figura 3: Esquema de salida para las instancias Mk01–Mk10 del benchmark Brandimarte. Cada fila representa una máquina; los bloques coloreados corresponden a operaciones asignadas por dos metaheurísticas distintas (MH1, MH2). El makespan $C_{\max}$ es el criterio a minimizar.

Tabla IV: Gap % del BKS por benchmark. Valores negativos indican superación de la cota de Mastrolilli & Gambardella [2]. En negrita: mejor valor de cada columna.

Metaheurística	edata	rdata	vdata	Brand.	Dauz.	Global
Memético	11.95	1.04	1.59	-12.84	6.32	-0.57
SA	12.62	2.39	1.88	-11.93	4.19	-0.31
AG	12.37	3.99	3.59	-12.54	5.15	0.17
TS+VNS	13.57	4.61	3.12	-7.94	7.67	2.36
PSO	13.34	6.78	5.26	-8.06	10.29	3.49
Tabú	16.57	5.18	2.70	-9.61	15.06	3.76
ACO	24.49	14.44	8.19	9.09	38.30	17.95

la cota reportada en la literatura, lo que ocurre especialmente en Brandimarte debido al avance del hardware en 25 años. La Fig. 6 resume el ranking global y la Fig. 4 muestra el detalle

completo por instancia.

VI-A. Resultados adicionales: HHS

La metaheurística HHS fue incorporada con posterioridad al diseño del protocolo experimental común. Sus experimentos se realizaron con un tiempo límite de 30 s por ejecución y cotas de referencia provenientes de una fuente distinta a Mastrolilli & Gambardella [2] —por ejemplo, $UB(Mk01) = 40$ y $UB(Mk10) = 193$ frente a los valores 42 y 296 del resto del grupo—, por lo que sus resultados no son directamente comparables. La Tabla III presenta el mejor makespan obtenido por cada metaheurística en cada instancia, y la Fig. 5 muestra las curvas de convergencia en tres instancias representativas.

VII. DISCUSIÓN

Los resultados de la Tabla IV y la Fig. 6 permiten ordenar las siete metaheurísticas de mejor a peor desempeño global. El Algoritmo Memético lidera con un gap global de -0.57% , superando en promedio las cotas de referencia de Mastrolilli & Gambardella [2]. El SA ocupa el segundo lugar (-0.31%), seguido por el AG ($+0.17\%$). El ACO queda claramente por debajo ($+17.95\%$), con dificultades especialmente marcadas en Dauzère ($+38.30\%$).

El mapa de calor de la Fig. 4 revela tres patrones consistentes:

1. **La flexibilidad facilita el problema.** En todas las metaheurísticas el gap disminuye al pasar de edata (baja flexibilidad) a vdata (alta flexibilidad).

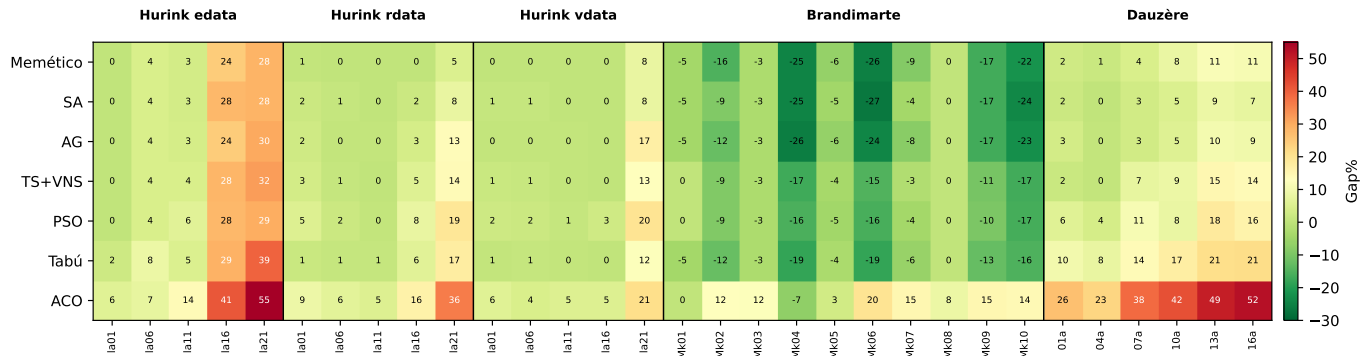


Figura 4: Mapa de calor del gap % por instancia y metaheurística. Verde = buen desempeño; rojo = gap elevado.

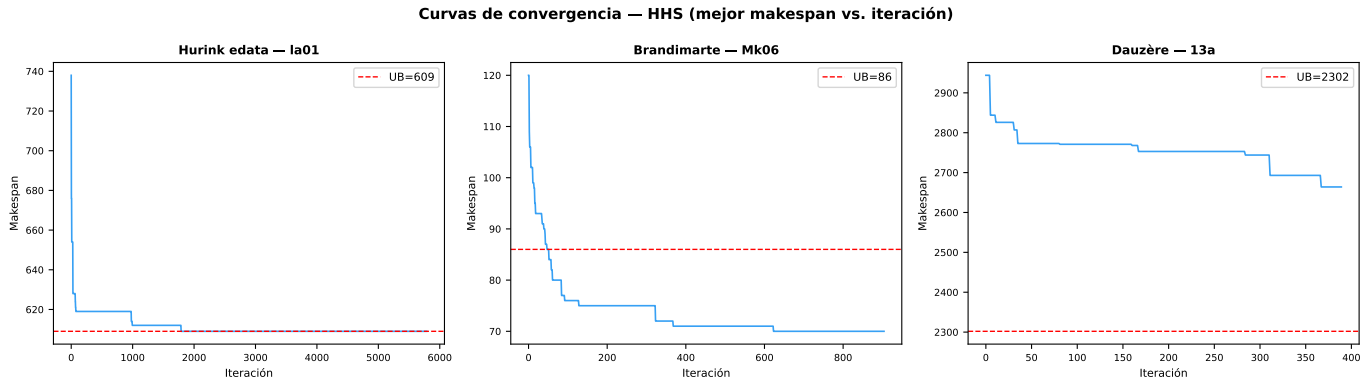


Figura 5: Curvas de convergencia del HHS en tres instancias representativas. La línea roja punteada indica la cota UB propia del HHS.

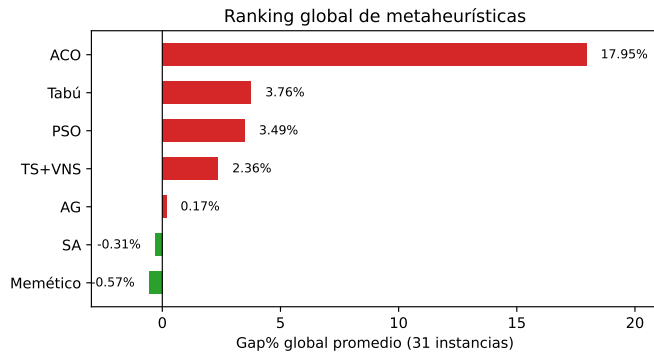


Figura 6: Ranking global por gap % promedio sobre las 31 instancias. Valores negativos indican superación de la cota de referencia.

- Los híbridos dominan.** El Memético y el TS+VNS figuran sistemáticamente entre los mejores. El SA también destaca gracias a su mecanismo de aceptación probabilística.
- Superación de cotas en Brandimarte.** Seis de las siete metaheurísticas obtienen gaps negativos, reflejo del avance del hardware en 25 años y la naturaleza de cota superior (no óptimo) de esos valores.

El caso del ACO (MMAS) sugiere que la calidad de la implementación y el ajuste de parámetros son tan determinantes como la elección del algoritmo.

Respecto al HHS, sus curvas de convergencia (Fig. 5) muestran convergencia rápida en instancias pequeñas, con dificultades en la21 y en las instancias más grandes de Dauzère.

VIII. CONCLUSIONES

Se implementaron y compararon siete metaheurísticas para el FJSP sobre 31 instancias de tres benchmarks clásicos, bajo un protocolo experimental uniforme. El Algoritmo Memético obtuvo el mejor desempeño global (gap % = -0,57), seguido por el SA (-0,31) y el AG (+0,17). Los dos primeros superaron en promedio las cotas de Mastrolilli & Gambardella [2]; el AG obtuvo un gap prácticamente nulo. El ACO presentó los resultados menos competitivos, en particular en Dauzère. Seis de los siete métodos superaron las cotas en Brandimarte, reflejo del avance del hardware desde el año 2000. La HHS, evaluada de forma independiente (30 s), mostró resultados prometedores en instancias pequeñas de Hurink y mayores dificultades a medida que crece el tamaño del problema.

Como trabajo futuro se plantea la comparación con un solver exacto (Gurobi o CPLEX) en instancias de menor tamaño, lo que permitiría medir la distancia al óptimo real.

DECLARACIÓN DE USO DE IA

En la elaboración de este trabajo se utilizaron herramientas de inteligencia artificial generativa (Claude, Anthropic) como apoyo en las siguientes tareas: generación y depuración de código Python para la implementación de las metaheurísticas, procesamiento y visualización de resultados experimentales, formateo del documento en L^AT_EX bajo la plantilla IEEE, y revisión ortográfica y gramatical del texto. El diseño algorítmico, la configuración de parámetros, la ejecución de experimentos y el análisis e interpretación de los resultados fueron realizados íntegramente por los autores.

REFERENCIAS

- [1] E. Hurink, B. Jurisch, and M. Thole, "Tabu search for the job-shop scheduling problem with multi-purpose machines," *OR Spektrum*, vol. 15, pp. 205–215, 1994.
- [2] M. Mastrolilli and L.M. Gambardella, "Effective neighborhood functions for the flexible job shop problem," *Journal of Scheduling*, vol. 3, no. 1, pp. 3–20, 2000.
- [3] P. Brandimarte, "Routing and scheduling in a flexible job shop by tabu search," *Annals of Operations Research*, vol. 41, pp. 157–183, 1993.
- [4] S. Dauzère-Pérès and J. Paulli, "An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search," *Annals of Operations Research*, vol. 70, pp. 281–306, 1997.
- [5] F. Glover, "Future paths for integer programming and links to artificial intelligence," *Computers & Operations Research*, vol. 13, no. 5, pp. 533–549, 1986.
- [6] N. Mladenović and P. Hansen, "Variable neighborhood search," *Computers & Operations Research*, vol. 24, no. 11, pp. 1097–1100, 1997.
- [7] J.W. Barnes and J.B. Chambers, "Flexible job shop scheduling by tabu search," Technical Report ORP96-09, Univ. of Texas at Austin, 1996.
- [8] M.R. Garey, D.S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Mathematics of Operations Research*, vol. 1, no. 2, pp. 117–129, 1976.
- [9] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W.H. Freeman, 1979.
- [10] J.H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [11] P. Moscato, "On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms," Tech. Rep. 826, Caltech Concurrent Computation Program, 1989.
- [12] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. ICNN'95*, Perth, 1995, pp. 1942–1948.
- [13] M. Dorigo, V. Maniezzo, and A. Colomi, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, 1996.
- [14] T. Stützle and H.H. Hoos, "MAX-MIN ant system," *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [15] S. Kirkpatrick, C.D. Gelatt, and M.P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [16] M. Mahdavi, M. Fesanghary, and E. Damangir, "An improved harmony search algorithm for solving optimization problems," *Applied Mathematics and Computation*, vol. 188, no. 2, pp. 1567–1579, 2007.
- [17] M.L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 4th ed. New York: Springer, 2012.
- [18] P. Brucker, *Scheduling Algorithms*, 5th ed. Berlin: Springer, 2007.
- [19] F. Pezzella, G. Morganti, and G. Ciaschetti, "A genetic algorithm for the flexible job-shop scheduling problem," *Computers & Operations Research*, vol. 35, no. 10, pp. 3202–3212, 2008.
- [20] W. Xia and Z. Wu, "An effective hybrid optimization approach for multi-objective flexible job-shop scheduling problems," *Computers & Industrial Engineering*, vol. 48, no. 2, pp. 409–425, 2005.
- [21] G. Zhang, X. Shao, P. Li, and L. Gao, "An effective hybrid particle swarm optimization algorithm for multi-objective flexible job-shop scheduling problem," *Computers & Industrial Engineering*, vol. 56, no. 4, pp. 1309–1318, 2009.
- [22] X. Li and L. Gao, "An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem," *Int. J. Production Economics*, vol. 174, pp. 93–110, 2016.
- [23] J. Dréo, A. Pétrowski, P. Siarry, and E. Taillard, *Metaheuristics for Hard Optimization*. Berlin: Springer, 2006.
- [24] E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, "Sequencing and scheduling: Algorithms and complexity," in *Handbooks in Operations Research*, vol. 4. Amsterdam: Elsevier, 1993, pp. 445–522.
- [25] G. Gatica and R.R. González, "Metaheurísticas aplicadas a problemas de scheduling en sistemas de manufactura flexible," *Ingeniare. Revista chilena de ingeniería*, vol. 19, no. 1, pp. 74–85, 2011.